

Salvador cardoso

Q1) For the two snippets of block-structure C code given below, indicate the values assigned to w, x, y & z .

a) `int w, x, y, z;
int i = 4, int j = 5;
{
 int j = 7;
 i = 6;`

<code>w = i + j;</code>	// $w = 13$
<code>x = i + j;</code>	// $x = 11$
<code>int i = 8;</code>	
<code>y = i + j;</code>	// $y = 13$
<code>z = i + j;</code>	// $z = 11$

~~$w = 13$~~
 ~~$x = 11$~~

b) `int w, x, y, z;
int i = 3, int j = 4;
{
 int i = 5;`

<code>w = i + j;</code>	// $w = 9$
<code>x = i + j;</code>	// $x = 7$
<code>int j = 6;</code>	
<code>i = 7;</code>	
<code>y = i + j;</code>	// $y = 13$
<code>z = i + j;</code>	// $z = 11$

Q2) For the block-structured code given below, assuming the usual static scoping of declarations, give the scope for each of the twelve declarations.

```

{ int w, x, y, z; /* Block B1 */
  { int x, z; /* Block B2 */
    { int w, x; /* Block B3 */
      { int w, x; /* Block B4 */
        { int y, z; /* Block B5 */
          {
            {
              {
                {

```

B1: $w \rightarrow \{B1, B2\}$

B1: $x \rightarrow \{B1\}$

B1: $y \rightarrow \{B1, B2, B3, B4\}$

B1: $z \rightarrow \{B1, B4\}$

B2: $x \rightarrow \{B2\}$

B2: $z \rightarrow \{B2, B3\}$

B3: $w \rightarrow \{B3\}$

B3: $x \rightarrow \{B3\}$

B4: $w \rightarrow \{B4, B5\}$

B4: $x \rightarrow \{B4, B5\}$

B5: $y \rightarrow \{B5\}$

B5: $z \rightarrow \{B5\}$

Q3) Assuming static type binding,

a) Explain the memory management for each line of the code from line 2 to line 8. If the procedure cannot compile, you should explain why & fix the code first.

b) what is the printout of this procedure?

c) Does this procedure have the problem of dangling pointer &/or memory leak? Explain?

```
Line 1: Procedure P: {  
line 2:   int *X := null;  
line 3:   X := 5; // incompatible types  
line 4:   int *y := new(10);  
line 5:   X := y;  
line 6:   print *X;  
line 7:   X := X + 2; // dangerous & UB because it points to a static int.  
line 8:   X := new(20);  
line 9:   print *y;  
line 10:  delete y;  
line 11: }
```

a) line 2 creates a stack-dynamic pointer variable
line 3 is illegal because X is a pointer while 5 is an integer. use new(5) instead;

line 4: y is a stack-dynamic variable, while
new(10) creates an explicit-heap dynamic variable

line 5: X & y point to the same heap-object containing 10.

line 6: dereferences X to print the data stored in the heap object.

line 7: This is pointer arithmetic. Because X (& y) point to an integer then you can increment the address of X, but you don't necessarily control this memory.

It is safer to do something like:

*X := *X + 2;

to increment the data 10 → 12

line 8: creates a new explicit-heap dynamic object for the integer 20

line 9: dereferences y to & print 12

line 10: deallocates the memory pointed by y.

b) depends on what the intention for line 7 was.
If you really wanted the VB of

$x = x + z;$

then the function prints 10 10.

If you wanted to increment the data pointed by x, then it prints 10 12

c) • After line 10, y is a dangling pointer, although the function ends immediately after.

• There are two memory leaks, from

- the heap object via line 3
- the heap object via line 8
- Memory ~~it~~ was heap-allocated, but not freed.

Q4) Consider the following Javascript skeletal program:

// The main program

var x;

function sub1() {

var x;

function sub2() {

...

}

} function sub3() {

...

}

Assume that the execution of this program is in the following unit order:

main calls sub1

sub1 calls sub2

sub2 calls sub3

a) Assuming static scoping, in the following, which declaration of x is the correct one for a reference to x ?

- i. sub1
- ii sub2
- iii sub3

b) repeat part a, but assume dynamic scoping.

a) As written, none of them are correct. sub3() & sub1() are siblings. When sub3() is called, it sees no declaration for x so it checks its ancestor scope, main. It sees a declaration of x there and references that.

b) under dynamic scoping, sub3() sees no declaration of x , so it checks sub2, then checks sub1() where it finds the declaration.

[Q8] Assume the following javascript program was interpreted using static-scoping rules. What value of x is displayed in function sub1? Under dynamic scoping rules, what value of x is displayed in function sub1?

```
var x;  
function sub1() {  
    document.write("x = " + x + " ");  
}  
function sub2() {  
    var x;  
    x = 10;  
    sub1();  
}  
x = 5;  
sub2();
```

Static scoping: $x = 5$;

dynamic scoping: $x = 10$;

Q6

consider the following C program:

```
void fun(void){
    int a,b,c; /* definition 1 */
    ...
    while(...){
        int b,c,d; /* definition 2 */
        ... ← 1
        while(...){
            int c,d,e; /* definition 3 */
            ... ← 2
        }
        ... ← 3
    }
    ... ← 4
}
```

For each of the marked points in this function, list each ~~variable~~ variable, along w/ the number of the definition statement that defines it.

1: a → def 1	2: a → def 1	3: a → def 1	4: a → def 1
b → def 2	b → def 2	b → def 2	b → def 1
c → def 2	c → def 3	c → def 2	c → def 1
d → def 2	d → def 3	d → def 2	
	c → def 3		

Q7

consider the following skeletal C program:

```
void fun1(void); /* prototype */
void fun2(void); /* prototype */
void fun3(void); /* prototype */
void main(){
    int a,b,c;
    ...
    void fun1(void){
        int b,c,d;
        ...
    }
    void fun2(void){
        int c,d,e;
        ...
    }
    void fun3(void){
        int d,e,f;
        ...
    }
}
```


Given the following calling sequences & assuming that dynamic scoping is used, what variables are visible during execution of the last function called? Include w/ each visible variable the name of the function in which it was defined.

a) main calls fun1; fun1 calls fun2; fun2 calls fun3

b) main calls fun1; fun1 calls fun3;

c) main calls fun2; fun2 calls fun3; fun3 calls fun1

d) main calls fun3; fun3 calls fun1;

e) main calls fun1; fun1 calls fun3; fun3 calls fun2

f) main calls fun3; fun3 calls fun2; fun2 calls fun1;

a) fun3() → {d, e, f}

fun2() → {c}

fun1() → {b}

main() → {a}

b) fun3() → {d, e, f}

fun1() → {b, c}

main() → {a}

c) fun1() → {b, c, d}

fun3() → {e, f}

fun2() → {}

main() → {a}

d) fun1() → {b, c, d}

fun3() → {e, f}

main() → {a}

e) fun2() → {c, d, e}

fun3() → {f}

fun1() → {b}

main() → {a}

f) fun1() → {b, c, d}

fun2() → {e}

fun3() → {f}

main() → {a}

108 The following is a grammar for regular expressions over symbols a & b only, using $\pm$ in place of $|$ for union, to avoid conflict w/ the use of vertical bar as a metasyMBOL in grammars:

$$\begin{aligned} r_{\text{expr}} &\rightarrow r_{\text{expr}} \pm r_{\text{term}} \mid r_{\text{term}} \\ r_{\text{term}} &\rightarrow r_{\text{term}} r_{\text{factor}} \mid r_{\text{factor}} \\ r_{\text{factor}} &\rightarrow r_{\text{factor}} * \mid r_{\text{primary}} \\ r_{\text{primary}} &\rightarrow a \mid b \end{aligned}$$

- left factor this grammar
- Does left factoring make the grammar suitable for top-down parsing?
- In addition to left factoring, eliminate left recursion from the original grammar.
- Is the resulting grammar suitable for top-down parsing?

- Grammar cannot be left-factored.
- No, still has left recursion
-

$$\begin{aligned} r_{\text{expr}} &\rightarrow r_{\text{term}} r_{\text{expr}}' \\ r_{\text{expr}}' &\rightarrow + r_{\text{term}} r_{\text{expr}}' \mid \epsilon \\ r_{\text{term}} &\rightarrow r_{\text{factor}} r_{\text{term}}' \\ r_{\text{term}}' &\rightarrow r_{\text{factor}} r_{\text{term}}' \mid \epsilon \\ r_{\text{factor}} &\rightarrow r_{\text{primary}} r_{\text{factor}}' \\ r_{\text{factor}}' &\rightarrow * r_{\text{factor}}' \mid \epsilon \\ r_{\text{primary}} &\rightarrow a \mid b \end{aligned}$$

- yes, this is sufficient for top-down parsing.
- verify via parsing table.

~~109~~

~~$S \rightarrow SS \mid SS^* \mid \epsilon$~~

$\text{first}(r_{\text{expr}}) = \{a, b\}$	$\text{follow}(r_{\text{expr}}) = \{\$, \}$
$\text{first}(r_{\text{expr}}') = \{+, \epsilon\}$	$\text{follow}(r_{\text{expr}}') = \{\$, \}$
$\text{first}(r_{\text{term}}) = \{a, b\}$	$\text{follow}(r_{\text{term}}) = \{+, \$, \}$
$\text{first}(r_{\text{term}}') = \{a, b, \epsilon\}$	$\text{follow}(r_{\text{term}}') = \{+, \$, \}$
$\text{first}(r_{\text{factor}}) = \{a, b\}$	$\text{follow}(r_{\text{factor}}) = \{a, b, +, \$, \}$
$\text{first}(r_{\text{factor}}') = \{*, \epsilon\}$	$\text{follow}(r_{\text{factor}}') = \{a, b, +, \$, \}$
$\text{first}(r_{\text{primary}}) = \{a, b\}$	$\text{follow}(r_{\text{primary}}) = \{*, a, b, +, \$, \}$

parsing table:

① $r_{\text{expr}} \rightarrow r_{\text{term}} r_{\text{expr}}'$					
② $r_{\text{expr}}' \rightarrow + r_{\text{term}} r_{\text{expr}}'$					
③ $r_{\text{expr}}' \rightarrow \epsilon$					
④ $r_{\text{term}} \rightarrow r_{\text{factor}} r_{\text{term}}'$	r_{expr}	a	b	+	*
⑤ $r_{\text{term}}' \rightarrow r_{\text{factor}} r_{\text{term}}'$	r_{expr}'	①	①	②	③ ③
⑥ $r_{\text{term}}' \rightarrow \epsilon$	r_{term}	4	4		
⑦ $r_{\text{factor}} \rightarrow r_{\text{primary}} r_{\text{factor}}$	r_{term}'	5	5	6	6
⑧ $r_{\text{factor}}' \rightarrow * r_{\text{factor}}'$	r_{factor}	7	7		
⑨ $r_{\text{factor}}' \rightarrow \epsilon$	r_{factor}'	9	9	9	9
⑩ $r_{\text{primary}} \rightarrow a b$	r_{primary}	10	10		

no conflicts: guaranteed to be LL(1)

Q9 $S \rightarrow SS+ | SS* | a$

a) $S \rightarrow SSA | a$
 $A \rightarrow + | *$

b) No, the grammar is still not suitable for top down parsing

c)
 $S \rightarrow aS'$
 $S' \rightarrow SAS' | \epsilon$
 $A \rightarrow + | *$

d) yes, this is suitable. verify w/ the parse table:

$\text{first}(S) = \{a\}$	$\text{follow}(S) = \{\$, +, *, \}$
$\text{first}(S') = \{a, \epsilon\}$	$\text{follow}(S') = \{\$, +, *, \}$
$\text{first}(A) = \{+, *\}$	$\text{follow}(A) = \{a, \$, +, *, \}$
	a + * \$
① $S \rightarrow aS'$	S ①
② $S' \rightarrow SAS'$	S' ② ③ 3 ③
③ $S' \rightarrow \epsilon$	A 4 4
④ $A \rightarrow + *$	

no conflicts: guaranteed to be LL(1)

Q10 $S \rightarrow 0S1 \mid 01$

a) $S \rightarrow 0A$
 $A \rightarrow S1 \mid 1$

b) I think so, but I need to verify w/ the parse table

c) cannot remove no left recursion to remove

d)

$\text{first}(S) = \{0\}$

$\text{follow}(S) = \{\$, 1\}$

$\text{first}(A) = \{1, 0\}$

$\text{follow}(A) = \{\$, 1\}$

	0	1	\$
S	1		
A	2	3	

1) $S \rightarrow 0A$

2) $A \rightarrow S1$

3) $A \rightarrow 1$

no conflict: guaranteed to be LL(1)

Q11

$\text{bexpr} \rightarrow \text{bexpr or bterm} \mid \text{bterm}$

$\text{bterm} \rightarrow \text{bterm and bfactor} \mid \text{bfactor}$

$\text{bfactor} \rightarrow \text{not bfactor} \mid (\text{bexpr}) \mid \text{true} \mid \text{false}$

a) cannot left factor

b) No, still has direct left recursion

c)

$\text{bexpr} \rightarrow \text{bterm bexpr}'$

$\text{bexpr}' \rightarrow \text{or bterm bexpr}' \mid \epsilon$

$\text{bterm} \rightarrow \text{bfactor bterm}'$

$\text{bterm}' \rightarrow \text{and bfactor bterm}' \mid \epsilon$

$\text{bfactor} \rightarrow \text{not bfactor} \mid (\text{bexpr}) \mid \text{true} \mid \text{false}$

d) I DK, need to check parse table.

$\text{first}(\text{bexpr}) = \{\text{not}, (, \text{true}, \text{false}\}$

$\text{follow}(\text{bexpr}) = \{\$,)\}$

$\text{first}(\text{bexpr}') = \{\text{or}, \epsilon\}$

$\text{follow}(\text{bexpr}') = \{\$,)\}$

$\text{first}(\text{bterm}) = \{\text{not}, (, \text{true}, \text{false}\}$

$\text{follow}(\text{bterm}) = \{\$,)\}$

$\text{first}(\text{bterm}') = \{\text{and}, \epsilon\}$

$\text{follow}(\text{bterm}') = \{\$,)\}$

$\text{first}(\text{bfactor}) = \{\text{not}, (, \text{true}, \text{false}\}$

$\text{follow}(\text{bfactor}) = \{\text{and}, \text{or}, \$,)\}$

- $bexpr \rightarrow bexpr \text{ or } bterm \mid bterm$
 ① $bexpr \rightarrow bterm \ bexpr'$
 2) $bexpr' \rightarrow \text{or } bterm \ bexpr'$
 3) $bexpr' \rightarrow \epsilon$
 4) $bterm \rightarrow bfactor \ bterm'$
 5) $bterm' \rightarrow \text{and } bfactor \ bterm'$
 6) $bterm' \rightarrow \epsilon$
 7) $bfactor \rightarrow \text{not } bfactor$
 8) $bfactor \rightarrow (bexp)$
 9) $bfactor \rightarrow \text{true}$
 10) $bfactor \rightarrow \text{false}$

	and	or	not	(	)	true	false	\$
$bexpr$			1	1		1	1	
$bexpr'$		2						3
$bterm$			4	4		4	4	
$bterm'$	5	6			6			6
$bfactor$			7	8		9	10	
$bfactor$								

no conflicts in the parsing table, so the grammar is LL(1)